

Laboratorium 3: Działania na liczbach

Opracowanie zestawu zadań (część teoretyczna)

Kacper Bednarczuk

Zadanie 1.1: Konwersja bazowa

a) 50

- **ZM (Znak-Moduł):** 00110010 (znak 0, moduł 50 = $32+16+2$)
- **U1 (Uzupełnienie do Jedności):** 00110010 (taki sam jak ZM dla liczb dodatnich)
- **U2 (Uzupełnienie do Dwóch):** 00110010 (taki sam jak ZM dla liczb dodatnich)

b) 80

- **ZM:** 01010000 (znak 0, moduł 80 = $64+16$)
- **U1:** 01010000
- **U2:** 01010000

c) -30

- **ZM:** 10011110 (znak 1, moduł 30 = $16+8+4+2 = 0011110$)
- **U1:** 11100001 (negacja bitów 00011110)
- **U2:** 11100010 ($U1 + 1$)

d) 100

- **ZM:** 01100100 (znak 0, moduł 100 = $64+32+4$)
 - **U1:** 01100100
 - **U2:** 01100100
-

Zadanie 1.2: Arytmetyka w U2

a) 50 + 80

- Operandy: 50 -> 00110010, 80 -> 01010000
- Dodawanie:

```
00110010 (50)
+ 01010000 (80)
```

10000010

- Wynik binarny: 10000010
- Wynik dziesiętny: -126 (ponieważ 10000010 w U2 to liczba ujemna, jej wartość to - (negacja(10000010) + 1) = - (01111101 + 1) = - (125+1) = -126)
- **Nadmiar (Overflow): Tak.** Suma dwóch liczb dodatnich dała wynik ujemny.

b) 50 – 30 (50 + (-30))

- Operandy: 50 -> 00110010 , -30 -> 11100010
- Dodawanie:

```
  00110010  (50)
+ 11100010  (-30)
-----
(1)00010100
```

- Wynik binarny: 00010100 (przeniesienie Cout=1 jest ignorowane)
- Wynik dziesiętny: 20
- **Nadmiar (Overflow): Nie.** Suma liczby dodatniej i ujemnej nigdy nie powoduje nadmiaru.

c) (-30) + (-100)

- 100 -> 01100100
- -100 (U2): negacja(01100100) + 1 = 10011011 + 1 = 10011100
- Operandy: -30 -> 11100010 , -100 -> 10011100
- Dodawanie:

```
  11100010  (-30)
+ 10011100  (-100)
-----
(1)01111110
```

- Wynik binarny: 01111110
- Wynik dziesiętny: 126
- **Nadmiar (Overflow): Tak.** Suma dwóch liczb ujemnych dała wynik dodatni.

d) 50 – (-30) (50 + 30)

- Operandy: 50 -> 00110010 , 30 -> 00011110
- Dodawanie:

```
  00110010  (50)
+ 00011110  (30)
-----
  01010000
```

- Wynik binarny: 01010000
- Wynik dziesiętny: 80
- **Nadmiar (Overflow): Nie.** Wynik 80 mieści się w zakresie 8-bitowego U2 [-128, 127].

Zadanie 1.3: Arytmetyka w ZM

a) 50 + 80

- Operandy: 50 -> 00110010 , 80 -> 01010000
- Opis: Obie liczby dodatnie, więc dodajemy moduły, znak wyniku dodatni.
- Operacja na modułach (7 bitów):

```

    0110010  (moduł 50)
+   1010000  (moduł 80)
-----
(1)0000010

```

- Wynik: 0 (znak) + 0000010 (moduł) = 0000010 . Wystąpiło przeniesienie z najstarszego bitu modułu (Cout=1), co sygnalizuje nadmiar.
- Wynik dziesiętny: 2 . Wynik jest niepoprawny, ponieważ 50+80=130 , co przekracza maksymalną wartość dla 7-bitowego modułu (127).

b) 50 – 30 (50 + (–30))

- Operandy: 50 -> 00110010 , -30 -> 10011110
- Opis: Liczby mają różne znaki, więc od większego modułu (50) odejmujemy mniejszy (30). Znak wyniku jest taki sam jak liczby o większym module (dodatni).
- Operacja na modułach:

```

    0110010  (moduł 50)
-   0011110  (moduł 30)
-----
    0010100

```

- Wynik: 0 (znak) + 0010100 (moduł) = 00010100
- Wynik dziesiętny: 20

c) (–30) + (–100)

- Operandy: -30 -> 10011110 , -100 -> 11100100
- Opis: Obie liczby ujemne, więc dodajemy moduły, znak wyniku ujemny.
- Operacja na modułach:

```

    0011110  (moduł 30)
+   1100100  (moduł 100)

```

(1)0000010

- Wynik: 1 (znak) + 0000010 (moduł) = 10000010 . Wystąpiło przeniesienie z najstarszego bitu modułu (Cout=1), co sygnalizuje nadmiar.
- Wynik dziesiętny: -2 . Wynik jest niepoprawny, ponieważ $30+100=130$, co przekracza maksymalną wartość dla 7-bitowego modułu (127).

d) 50 – (-30) (50 + 30)

- Operandy: 50 -> 00110010 , 30 -> 00011110
- Opis: Obie liczby dodatnie, więc dodajemy moduły, znak wyniku dodatni.
- Operacja na modułach:

```
  0110010  (moduł 50)
+ 0011110  (moduł 30)
-----
  1010000
```

- Wynik: 0 (znak) + 1010000 (moduł) = 01010000
- Wynik dziesiętny: 80

Zadanie 1.4: Arytmetyka w U1

a) 50 + 80

- Operandy: 50 -> 00110010 , 80 -> 01010000
- Dodawanie:

```
  00110010
+ 01010000
-----
  10000010
```

- Przeniesienie Cout=0 . Korekcja nie jest potrzebna.
- Wynik binarny: 10000010
- Wynik dziesiętny: -125 (w U1 10000010 to negacja 01111101 , czyli -125)
- Wystąpił nadmiar (suma dwóch dodatnich liczb dała wynik ujemny).

b) 50 – 30 (50 + (-30))

- Operandy: 50 -> 00110010 , -30 -> 11100001
- Dodawanie:

```
  00110010
+ 11100001
```

```
-----  
(1)00010011
```

- Przeniesienie Cout=1 . Należy dodać 1 do wyniku (korekcja).
- $00010011 + 1 = 00010100$
- Wynik binarny: 00010100
- Wynik dziesiętny: 20

c) (-30) + (-100)

- -100 (U1): negacja(01100100) = 10011011
- Operandy: -30 -> 11100001 , -100 -> 10011011
- Dodawanie:

```
  11100001  
+ 10011011  
-----  
(1)01111100
```

- Przeniesienie Cout=1 . Należy dodać 1 do wyniku.
- $01111100 + 1 = 01111101$
- Wynik binarny: 01111101
- Wynik dziesiętny: 125
- Wystąpił nadmiar (suma dwóch ujemnych liczb dała wynik dodatni).

d) 50 - (-30) (50 + 30)

- Operandy: 50 -> 00110010 , 30 -> 00011110
- Dodawanie:

```
  00110010  
+ 00011110  
-----  
  01010000
```

- Przeniesienie Cout=0 . Korekcja nie jest potrzebna.
- Wynik binarny: 01010000
- Wynik dziesiętny: 80

Zadanie 1.5: Pytania porównawcze

1. Najprostszy system dla układu cyfrowego:

System **U2 (Uzupełnienie do Dwóch)** jest najprostszy w implementacji sprzętowej.

Uzasadnienie:

- **Jedna operacja:** Zarówno dodawanie, jak i odejmowanie sprowadzają się do tej samej operacji dodawania binarnego. Odejmowanie $A - B$ jest realizowane jako $A + (-B)$, gdzie $-B$ to liczba przeciwna do B w kodzie U2.
- **Brak analizy znaków:** Układ (np. sumator ALU) nie musi analizować znaków operandów przed wykonaniem działania, w przeciwieństwie do systemu ZM, który wymaga osobnych algorytmów dla różnych kombinacji znaków.
- **Brak korekcji:** W przeciwieństwie do systemu U1, w U2 nie ma potrzeby wykonywania dodatkowego kroku korekcyjnego (dodawania przeniesienia cyklicznego), co upraszcza i przyspiesza działanie układu.

2. Problem podwójnego zera:

Problem podwójnego zera występuje w systemach **ZM (Znak-Moduł)** oraz **U1 (Uzupełnienie do Jedności)**.

- W ZM: 00000000 (+0) i 10000000 (-0).
- W U1: 00000000 (+0) i 11111111 (-0).

Problem ten komplikuje operacje porównywania (np. sprawdzania, czy $A = B$), ponieważ układ cyfrowy musi sprawdzić dwie różne reprezentacje binarne, aby stwierdzić, czy liczba jest zerem. Proste porównanie bit po bicie ($A \text{ XOR } B$) nie wystarczy, gdyż $+0$ i -0 dałyby wynik różny od zera, mimo że matematycznie są sobie równe. Wymaga to dodatkowej logiki w układach porównujących, co zwiększa ich złożoność. System U2 unika tego problemu, posiadając tylko jedną, jednoznaczną reprezentację zera (00000000).